

AMT Horizon

Functional Specification / Technical Specification Document

DOCUMENT VERSION 2.0

19/01/2026

Author

Name	Role
Alexis SANTOS	Program Manager / Tech Lead

Revision History

Date	Version	Description of Changes
14/01/2026	1.0	- Separated Functional and Technical Specifications into distinct documents. - Added detailed Database Structure and API Endpoints sections.
14/01/2026	1.3	- Prioritized actions over API endpoints. - Added detailed comments and explanations for tables and enums.

Date	Version	Description of Changes
19/01/2026	2.0	• Added Entity Relationship Diagram (ERD) with Mermaid showing all 20 tables and relationships. • Added 3 missing tables (messages, two_factor_auth, payment_transactions). • Added foreign key constraints with ON DELETE actions to all tables. • Added CHECK constraints for data validation (ratings 1-5, date ranges, sender/recipient validation). • Added UNIQUE constraints to prevent duplicates (email, ride-customer pairs). • Added comprehensive index strategy (15+ single-column and composite indexes, including GIST for geospatial queries). • Added detailed explanations for indexes and constraints with performance comparisons. • Added Drizzle ORM query examples section alongside SQL examples. • Added Database Migration Strategy with Drizzle Kit workflow, rollback procedures, and best practices. • Added Background Jobs & Scheduled Tasks section with Vercel Cron implementation (6 scheduled tasks). • Added Performance Optimization section (N+1 prevention, pagination, caching strategies). • Added Webhook Handling section (Stripe integration with signature verification). • Enhanced Code Actions with Zod validation schemas, error codes, and ActionResponse type. • Added transaction examples for payment processing and driver assignment. • Updated Business Logic diagrams with Mermaid flowcharts. • Enhanced Deployment section with environment variables, CI/CD pipeline, and monitoring. • Updated Table of Contents to reflect all new sections.

► Table of Contents

- [I. Introduction](#)
 - [A. Document Purpose](#)
 - [B. Target Audience](#)
 - [C. Technologies Used](#)
- [II. Overall Architecture](#)
 - [A. Architecture Diagram](#)
 - [B. Data Flow](#)
- [III. Database Structure \(PostgreSQL/Neon\)](#)
 - [A. Conceptual Schema](#)
 - [Entity Relationship Diagram \(ERD\)](#)
 - [Key Tables and Their Purpose](#)
 - [1. Users Table](#)

- 2. Rides Table
 - 3. Ride Customers Table
 - 4. Ride Options Table
 - 5. Ride Selected Options Table
 - 6. Assignment Requests Table
 - 7. Invoices Table
 - 8. Notifications Table
 - 9. Notification Preferences Table
 - 10. Activity Logs Table
 - 11. Shift Planning Table
 - 12. Productions Table
 - 13. Projects Table
 - 14. Account Table
 - 15. Session Table
 - 16. Verification Table
 - 17. Ratings Table
 - 18. Messages Table
 - 19. Two Factor Auth Table
 - 20. Payment Transactions Table
- B. Enums Types
 - Key Enums and Their Purpose
 - 1. User Role Enum
 - 2. Ride Status Enum
 - 3. Request Status Enum
 - 4. Invoice Status Enum
 - 5. Shift Status Enum
 - 6. Option Type Enum
 - 7. Vehicle Type Enum
- C. Indexes and Constraints
 - Single-Column Indexes
 - Composite Indexes
 - Summary Table
- D. Explanations
 - Tables
 - Enums
 - Indexes and Constraints
 - Why Indexes Are Useful
 - Why Constraints Are Useful
 - Performance Considerations
- IV. SQL Query Examples
 - A. List unassigned rides
 - B. Generate an invoice
 - C. Update assignment request status
 - D. Retrieve user's unread notifications
 - E. List available drivers within a 5 km radius
 - F. Calculate driver's average rating

- G. Send invoice via application
- H. List invoices eligible for reminder (unpaid, overdue)
- I. Send manual reminder
- J. Drizzle ORM Query Examples
 - List unassigned rides
 - Find available drivers within 5 km radius
 - Retrieve user's unread notifications
 - Calculate driver's average rating
 - Get ride with full details (including customers and driver)
 - List invoices eligible for reminder
- K. Database Migration Strategy
 - Overview
 - Migration Workflow
 - Step-by-Step Process
 - 1. Generate Migration
 - 2. Review Migration SQL
 - 3. Apply Migration
 - 4. Rollback Strategy
 - Migration Best Practices
 - drizzle.config.ts
 - Migration Checklist
- V. Code Actions (Server Actions and Components)
 - A. Code Action Structure
 - Example: Create a Ride
 - B. Main Code Actions
 - C. API Response Format
 - D. Validation Schemas
 - E. Error Handling
 - F. Transactions
 - Additional Transaction Examples
- VI. Business Logic
 - A. Key Algorithms
 - 1. Automatic Driver Assignment
 - 2. Price Calculation
 - 3. Invoice Management
 - B. Business Logic in Code Actions
 - Example: Ride Price Calculation
- VII. Authentication & Security
 - A. Authentication
 - B. Data Security
- VIII. External Integrations
 - A. Geolocation (Mapbox/Google Maps)
 - B. Payments (Stripe/PayPal)
 - C. OCR (Tesseract.js)
 - D. Rate Limiting
 - E. Caching Strategy

- F. File Storage & OCR
 - G. Real-time Communication
 - H. Background Jobs & Scheduled Tasks
 - Scheduled Tasks Overview
 - Implementation with Vercel Cron
 - Example: Invoice Reminder Job
 - Example: Activity Log Cleanup
 - I. Performance Optimization
 - Query Optimization
 - Connection Pooling
 - Caching Strategies
 - J. Webhook Handling
 - Stripe Webhook Integration
- IX. Testing Strategy
 - A. Unit Tests (Vitest)
 - B. Integration Tests
 - C. End-to-End Tests
- X. Deployment & CI/CD
 - A. Environment Variables
 - B. Environments
 - C. CI/CD Pipeline
 - D. Rollback Strategy
 - E. Monitoring & Logging

I. Introduction

A. Document Purpose

This document outlines the **technical architecture** of **AMT Horizon**, focusing on:

- Database structure and optimizations.
- **Code Actions** (NextJS Server Actions) for direct database interaction.
- Business logic, external integrations, and security.
- Testing, deployment, and best practices.

B. Target Audience

Developers, Tech Leads, QA engineers, and stakeholders involved in implementation or maintenance.

C. Technologies Used

Technology	Role
NextJS	Frontend and backend framework (App Router, Server Components).

Technology	Role
Drizzle ORM	Type-safe ORM for PostgreSQL/Neon interactions.
Neon	Serverless PostgreSQL database.
better-auth	Authentication and session management.
Vitest	Unit and integration testing.
Zod	Runtime data validation for inputs before processing by Drizzle.
Mapbox/Google Maps	Geolocation and distance calculation.
Stripe/PayPal	Online payments.
Tesseract.js	OCR for text extraction from images (tickets, rides).

II. Overall Architecture

A. Architecture Diagram



[!Note]

- The application uses NextJS App Router with Server Components.
- Zod validates inputs before Drizzle processes them.
- All database interactions go through Drizzle ORM.
- External services are integrated via server actions.

B. Data Flow

flowchart LR

```

A[User Interface] --> B[User Interaction]
B -- Call --> C[Server Actions]
C --> D[Zod Validation]
D -- Validated Data --> E[Drizzle ORM]
E -- Query/Mutation --> F[(Neon Database)]
F -- Return Data/Confirm --> E
E --> C
C -- Update --> A[NextJS Updates UI]

```

[!Note]

- Server Actions replace traditional API endpoints.
- **Zod validates inputs** before Drizzle processes them.
- UI updates are handled reactively by NextJS.

III. Database Structure (PostgreSQL/Neon)

A. Conceptual Schema

The database schema for AMT Horizon is designed to support a comprehensive taxi management system. Each table is tailored to handle specific aspects of the application, from user management to ride tracking, billing, and notifications.

Entity Relationship Diagram (ERD)

erDiagram

```

users ||--o{ rides : "drives"
users ||--o{ ride_customers : "books"
users ||--o{ assignment_requests : "requests"
users ||--o{ ratings : "rates"
users ||--o{ notifications : "receives"
users ||--o| notification_preferences : "has"
users ||--o{ activity_logs : "performs"
users ||--o{ session : "has"
users ||--o{ account : "has"
users ||--o| two_factor_auth : "has"
users ||--o{ messages : "sends"
users ||--o{ messages : "receives"
users ||--o{ shiftPlanning : "schedules"

rides ||--o{ ride_customers : "includes"
rides ||--o{ assignment_requests : "receives"
rides ||--o{ rideSelectedOptions : "has"
rides ||--|{ invoices : "generates"
rides ||--o{ ratings : "receives"
rides ||--o{ messages : "relates_to"
rides }o--|| productions : "belongs_to"
rides }o--|| projects : "belongs_to"

```

```
productions ||--o{ projects : "contains"
productions ||--o{ rides : "books"
productions ||--o{ shiftPlanning : "schedules"

projects ||--o{ rides : "includes"
projects ||--o{ shiftPlanning : "schedules"

invoices ||--o{ payment_transactions : "has"

users {
    text id PK
    text name
    text email UK
    text role
    decimal latitude
    decimal longitude
    text vehicle_type
    boolean available
}

rides {
    serial id PK
    text driver_id FK
    text production FK
    text project FK
    varchar departure
    varchar destination
    timestamp departure_time
    decimal price
    text status
}

ride_customers {
    serial id PK
    integer ride_id FK
    text customer_id FK
}

assignment_requests {
    serial id PK
    integer ride_id FK
    text driver_id FK
    text status
}

invoices {
    serial id PK
    integer ride_id FK
    decimal total
    decimal paid_amount
    varchar currency
    text status
}
```



```
payment_transactions {
  serial id PK
  integer invoice_id FK
  decimal amount
  varchar payment_method
  text status
}

ratings {
  serial id PK
  integer ride_id FK
  text customer_id FK
  text driver_id FK
  integer rating
}

messages {
  serial id PK
  text sender_id FK
  text recipient_id FK
  integer ride_id FK
  text message
}

productions {
  text id PK
  text name
  text contact_email
}

projects {
  text id PK
  text production_id FK
  text name
  timestamp start_date
  timestamp end_date
}
```

[!Note]

- Cardinality notation: `||` (one), `o{` (zero or many), `|{` (one or many)
- All foreign keys have appropriate ON DELETE actions defined
- User roles determine which relationships are active (e.g., drivers have vehicles, customers book rides)
- The diagram shows the logical relationships; actual implementation uses Drizzle ORM relations

Key Tables and Their Purpose

1. Users Table

-- Users Table: Stores all user accounts with roles, verification status, and vehicle details for drivers.

```
CREATE TABLE "users" (  
  "id" text PRIMARY KEY NOT NULL,  
  "name" text NOT NULL,  
  "email" text NOT NULL,  
  "email_verified" boolean DEFAULT false NOT NULL,  
  "image" text,  
  "created_at" timestamp DEFAULT now() NOT NULL,  
  "updated_at" timestamp DEFAULT now() NOT NULL,  
  "role" "user_role",  
  "banned" boolean DEFAULT false,  
  "ban_reason" text,  
  "ban_expires" timestamp,  
  -- Location fields  
  "latitude" numeric(9, 6),  
  "longitude" numeric(9, 6),  
  "last_location_update" timestamp,  
  -- Driver-specific fields  
  "vehicle_type" "vehicle_type",  
  "vehicle_plate" text,  
  "vehicle_model" text,  
  "vehicle_color" text,  
  "available" boolean DEFAULT true, -- column to indicate driver  
  availability  
  
  CONSTRAINT "users_email_unique" UNIQUE("email")  
);
```

Purpose: Stores user accounts, roles, location data, verification status, and drivers-specific details.

[!Note]

- The **available** column indicates whether a driver is currently available for ride assignments.
- Driver-specific fields (**vehicle_type**, **vehicle_plate**, etc.) are only populated for users with the **driver** role.

2. Rides Table

-- Rides Table: Stores ride details, including departure, destination, status, and associated options.

```
CREATE TABLE "rides" (  
  "id" serial PRIMARY KEY NOT NULL,  
  "departure" varchar(255) NOT NULL,  
  "destination" varchar(255) NOT NULL,  
  "departure_time" timestamp NOT NULL,  
  "arrival_time" timestamp,  
  "distance_km" numeric(10, 2),  
  "price" numeric(10, 2) NOT NULL,  
  "status" "ride_status" DEFAULT 'pending' NOT NULL,
```

```
"photo_url" varchar(512),
"created_at" timestamp DEFAULT now() NOT NULL,
"driver_id" text REFERENCES users(id) ON DELETE SET NULL,
"customer_notes" text,
"production" text REFERENCES productions(id) ON DELETE SET NULL,
"project" text REFERENCES projects(id) ON DELETE SET NULL,
"waiting_time" integer DEFAULT 0,
"options" jsonb DEFAULT '[]'::jsonb
);
```

Purpose: Stores ride details.

[!Note]

- The `options` field stores selected ride options as JSON for flexibility.
- `waiting_time` is measured in minutes and affects the final price calculation.

3. Ride Customers Table

```
-- Ride Customers Table: Links rides to customers, allowing multiple
customers per ride.
CREATE TABLE "ride_customers" (
  "id" serial PRIMARY KEY NOT NULL,
  "ride_id" integer NOT NULL REFERENCES rides(id) ON DELETE CASCADE,
  "customer_id" text NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  "created_at" timestamp DEFAULT now() NOT NULL,
  CONSTRAINT "ride_customers_unique" UNIQUE("ride_id", "customer_id")
);
```

Purpose: Links rides to customers, allowing multiple customers per ride. This supports group rides and shared billing.

4. Ride Options Table

```
-- Ride Options Table: Defines available ride options (e.g., VIP, baby
seat) and their additional prices.
CREATE TABLE "ride_options" (
  "id" serial PRIMARY KEY NOT NULL,
  "name" "option_type" NOT NULL,
  "description" text,
  "additional_price" numeric(10, 2) DEFAULT '0' NOT NULL
);
```

Purpose: Defines available ride options (e.g., VIP, baby seat) and their additional prices. This allows for customizable ride experiences.

5. Ride Selected Options Table

```
-- Ride Selected Options Table: Links rides to selected options, storing
the price at the time of selection.
CREATE TABLE "rideSelectedOptions" (
  "id" serial PRIMARY KEY NOT NULL,
  "ride_id" integer NOT NULL REFERENCES rides(id) ON DELETE CASCADE,
  "option_name" "option_type" NOT NULL,
  "price" numeric(10, 2) NOT NULL
);
```

Purpose: Links rides to selected options, storing the price at the time of selection. This ensures accurate billing for optional services.

6. Assignment Requests Table

```
-- Assignment Requests Table: Tracks driver requests to be assigned to
rides.
CREATE TABLE "assignment_requests" (
  "id" serial PRIMARY KEY NOT NULL,
  "ride_id" integer NOT NULL REFERENCES rides(id) ON DELETE CASCADE,
  "driver_id" text NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  "status" "request_status" DEFAULT 'pending' NOT NULL,
  "requested_at" timestamp DEFAULT now() NOT NULL,
  CONSTRAINT "assignment_requests_unique" UNIQUE("ride_id", "driver_id")
);
```

Purpose: Tracks driver requests to be assigned to rides. This supports the dynamic assignment process.

[!Note]

- Drivers can have a maximum of 5 simultaneous pending requests.
- Requests are automatically rejected if the ride is assigned to another driver.

7. Invoices Table

```
-- Invoices Table: Stores invoice details, including ride associations,
amounts, and payment status.
CREATE TABLE "invoices" (
  "id" serial PRIMARY KEY NOT NULL,
  "ride_id" integer NOT NULL REFERENCES rides(id) ON DELETE CASCADE,
  "waiting_fee" numeric(10, 2) DEFAULT '0' NOT NULL,
  "sub_total" numeric(10, 2) NOT NULL,
  "tax" numeric(10, 2) DEFAULT '0' NOT NULL,
  "total" numeric(10, 2) NOT NULL,
  "paid_amount" numeric(10, 2) DEFAULT '0' NOT NULL,
  "remaining_amount" numeric(10, 2) GENERATED ALWAYS AS (total -
paid_amount) STORED,
  "currency" varchar(3) DEFAULT 'EUR' NOT NULL,
```

```

    "status" "invoice_status" DEFAULT 'unpaid' NOT NULL,
    "invoice_date" timestamp DEFAULT now() NOT NULL,
    "due_date" timestamp DEFAULT CURRENT_DATE + INTERVAL '30 days' NOT
NULL,
    "pdf_url" varchar(512),
    "sent_via_app" boolean DEFAULT false NOT NULL,
    "app_sent_at" timestamp,
    "reminders_sent" integer DEFAULT 0 NOT NULL,
    "last_reminder_message" text
);

```

Purpose: Tracks invoice details, billing amounts, and payment status.

[!Note]

- The `due_date` defaults to 30 days after invoice creation.
- Reminders are sent at D+14, D+21, and D+30 after the due date (maximum 3 reminders).
- The `sent_via_app` field indicates whether the invoice was delivered through the application.
- Partial payments are supported: `paid_amount` tracks payments, `remaining_amount` is auto-calculated.
- `currency` defaults to 'EUR' but supports internationalization (ISO 4217 codes).

8. Notifications Table

```

-- Notifications Table: Stores notifications sent to users, with read
status.
CREATE TABLE "notifications" (
    "id" serial PRIMARY KEY NOT NULL,
    "user_id" text NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    "message" text NOT NULL,
    "type" varchar(50) DEFAULT 'info' NOT NULL,
    "is_read" boolean DEFAULT false NOT NULL,
    "created_at" timestamp DEFAULT now() NOT NULL
);

```

Purpose: Stores notifications sent to users, with read status. This supports real-time communication and alerts.

9. Notification Preferences Table

```

-- Notification Preferences Table: Stores user preferences for
notification channels.
CREATE TABLE "notification_preferences" (
    "id" serial PRIMARY KEY NOT NULL,
    "user_id" text NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    "email" boolean DEFAULT true NOT NULL,
    "push" boolean DEFAULT true NOT NULL,
    "sms" boolean DEFAULT false NOT NULL,

```

```
    "created_at" timestamp DEFAULT now() NOT NULL,  
    CONSTRAINT "notification_preferences_unique" UNIQUE("user_id")  
);
```

Purpose: Stores user preferences for notification channels. This allows users to customize how they receive alerts.

10. Activity Logs Table

```
-- Activity Logs Table: Logs user actions for auditing and compliance.  
CREATE TABLE "activity_logs" (  
    "id" serial PRIMARY KEY NOT NULL,  
    "user_id" text NOT NULL REFERENCES users(id) ON DELETE SET NULL,  
    "action" varchar(255) NOT NULL,  
    "resource_type" varchar(100),  
    "resource_id" text,  
    "details" text,  
    "ip_address" varchar(45),  
    "created_at" timestamp DEFAULT now() NOT NULL  
);
```

Purpose: Logs user actions for auditing and compliance. This is essential for security and GDPR compliance.

[!Note]

- Activity logs are retained for 6 months in compliance with GDPR.
- Logged actions include ride creation, assignment, invoice generation, and account modifications.

11. Shift Planning Table

```
-- Shift Planning Table: Manages driver shift schedules, linking to  
productions and projects.  
CREATE TABLE "shiftPlanning" (  
    "id" serial PRIMARY KEY NOT NULL,  
    "driver_id" text NOT NULL REFERENCES users(id) ON DELETE CASCADE,  
    "start_time" timestamp NOT NULL,  
    "end_time" timestamp NOT NULL,  
    "status" "shift_status" DEFAULT 'planned',  
    "production_id" text REFERENCES productions(id) ON DELETE SET NULL,  
    "project_id" text REFERENCES projects(id) ON DELETE SET NULL,  
    CONSTRAINT "shift_planning_time_check" CHECK (end_time > start_time)  
);
```

Purpose: Manages driver shift schedules, linking to productions and projects. This supports workforce management and scheduling.

[!Note]

- The `status` field tracks shift lifecycle: planned → active → completed/cancelled.
- Shifts must be linked to a driver but can optionally be associated with productions/projects.
- Time constraint ensures `end_time` is always after `start_time`.

12. Productions Table

```
-- Productions Table: Stores production details, such as address and
contact information.
CREATE TABLE "productions" (
  "id" text PRIMARY KEY NOT NULL,
  "name" text NOT NULL,
  "address" text,
  "contact_name" text,
  "contact_email" text NOT NULL,
  "contact_phone" text,
  "created_at" timestamp DEFAULT now() NOT NULL,
  CONSTRAINT "productions_email_check" CHECK (contact_email ~* '^[A-Za-
z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$')
);
```

Purpose: Stores production details, such as address and contact information. This supports project and production management.

[!Note]

- Productions represent companies or organizations that book rides for their employees.
- Email validation ensures `contact_email` is properly formatted.
- Productions can have multiple associated projects.

13. Projects Table

```
-- Projects Table: Stores project details, linked to productions.
CREATE TABLE "projects" (
  "id" text PRIMARY KEY NOT NULL,
  "name" text NOT NULL,
  "production_id" text NOT NULL REFERENCES productions(id) ON DELETE
CASCADE,
  "start_date" timestamp NOT NULL,
  "end_date" timestamp NOT NULL,
  "created_at" timestamp DEFAULT now() NOT NULL,
  CONSTRAINT "projects_date_check" CHECK (end_date > start_date)
);
```

Purpose: Stores project details, linked to productions. This supports project tracking and management.

[!Note]

- Projects are time-bound initiatives within a production.
- Date constraint ensures end_date is after start_date.
- Deleting a production cascades to delete all its projects.

14. Account Table

```
-- Account Table: Manages user authentication accounts and tokens.
CREATE TABLE "account" (
  "id" text PRIMARY KEY NOT NULL,
  "account_id" text NOT NULL,
  "provider_id" text NOT NULL,
  "user_id" text NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  "access_token" text,
  "refresh_token" text,
  "id_token" text,
  "access_token_expires_at" timestamp,
  "refresh_token_expires_at" timestamp,
  "scope" text,
  "password" text,
  "created_at" timestamp DEFAULT now() NOT NULL,
  "updated_at" timestamp NOT NULL
);
```

Purpose: Manages user authentication accounts and tokens. This is essential for secure user authentication.

[!Note]

- Managed by better-auth for OAuth and credential-based authentication.
- Supports multiple providers (Google, email/password, etc.).
- Tokens are encrypted at rest for security.

15. Session Table

```
-- Session Table: Manages user sessions, including tokens and IP
addresses.
CREATE TABLE "session" (
  "id" text PRIMARY KEY NOT NULL,
  "expires_at" timestamp NOT NULL,
  "token" text NOT NULL,
  "created_at" timestamp DEFAULT now() NOT NULL,
  "updated_at" timestamp NOT NULL,
  "ip_address" text,
  "user_agent" text,
  "user_id" text NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  "impersonated_by" text REFERENCES users(id) ON DELETE SET NULL,
  CONSTRAINT "session_token_unique" UNIQUE("token")
);
```


Purpose: Manages user sessions, including tokens and IP addresses. This supports session management and security.

[!Note]

- Sessions expire based on the `expires_at` timestamp.
- `impersonated_by` supports admin impersonation for support purposes.
- Tokens are hashed for security.

16. Verification Table

```
-- Verification Table: Stores verification tokens for email and other
validations.
CREATE TABLE "verification" (
  "id" text PRIMARY KEY NOT NULL,
  "identifier" text NOT NULL,
  "value" text NOT NULL,
  "expires_at" timestamp NOT NULL,
  "created_at" timestamp DEFAULT now() NOT NULL,
  "updated_at" timestamp DEFAULT now() NOT NULL
);
```

Purpose: Stores verification tokens for email and other validations. This supports user verification processes.

[!Note]

- Used for email verification, password reset, and 2FA setup.
- Tokens expire after 24 hours (configurable via `expires_at`).
- Tokens are single-use and deleted after successful verification.

17. Ratings Table

```
-- Ratings Table: Stores customer ratings and comments for rides.
CREATE TABLE "ratings" (
  "id" serial PRIMARY KEY NOT NULL,
  "ride_id" integer NOT NULL REFERENCES rides(id) ON DELETE CASCADE,
  "customer_id" text NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  "driver_id" text NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  "rating" integer NOT NULL CHECK (rating >= 1 AND rating <= 5),
  "comment" text,
  "created_at" timestamp DEFAULT now() NOT NULL,
  CONSTRAINT "ratings_unique" UNIQUE("ride_id", "customer_id")
);
```

Purpose: Stores customer ratings and comments for rides. This supports quality control and driver performance tracking.

[!Note]

- Ratings range from 1 to 5 stars (enforced by CHECK constraint).
- Comments are optional but help provide context for the rating.
- Ratings are linked to specific rides and cannot be duplicated per customer.
- Driver ID is stored for performance analytics.

18. Messages Table

```
-- Messages Table: Stores chat messages between users (drivers and customers).
CREATE TABLE "messages" (
  "id" serial PRIMARY KEY NOT NULL,
  "sender_id" text NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  "recipient_id" text NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  "ride_id" integer REFERENCES rides(id) ON DELETE SET NULL,
  "message" text NOT NULL,
  "is_read" boolean DEFAULT false NOT NULL,
  "created_at" timestamp DEFAULT now() NOT NULL,
  CONSTRAINT "messages_check" CHECK (sender_id != recipient_id)
);
```

Purpose: Stores in-app chat messages between drivers and customers for ride coordination.

[!Note]

- Messages are retained for 30 days as per FSD requirements.
- Messages must be between different users (enforced by CHECK constraint).
- Optional link to rides allows context-based messaging.

19. Two Factor Auth Table

```
-- Two Factor Auth Table: Stores 2FA secrets and backup codes for users.
CREATE TABLE "two_factor_auth" (
  "id" serial PRIMARY KEY NOT NULL,
  "user_id" text NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  "secret" text NOT NULL,
  "backup_codes" text[] NOT NULL,
  "enabled" boolean DEFAULT false NOT NULL,
  "verified_at" timestamp,
  "created_at" timestamp DEFAULT now() NOT NULL,
  "updated_at" timestamp DEFAULT now() NOT NULL,
  CONSTRAINT "two_factor_auth_unique" UNIQUE("user_id")
);
```

Purpose: Manages two-factor authentication for users (mandatory for admins, optional for others).

[!Note]

- `secret` is encrypted and used for TOTP generation.
- `backup_codes` are hashed one-time use codes for account recovery.
- `enabled` indicates whether 2FA is active for the user.
- Required for admins as per security requirements.

20. Payment Transactions Table

```
-- Payment Transactions Table: Records all payment attempts and their
outcomes.
CREATE TABLE "payment_transactions" (
  "id" serial PRIMARY KEY NOT NULL,
  "invoice_id" integer NOT NULL REFERENCES invoices(id) ON DELETE
CASCADE,
  "amount" numeric(10, 2) NOT NULL,
  "currency" varchar(3) DEFAULT 'EUR' NOT NULL,
  "payment_method" varchar(50) NOT NULL,
  "provider" varchar(50) NOT NULL,
  "provider_transaction_id" text,
  "status" varchar(20) NOT NULL,
  "error_message" text,
  "metadata" jsonb DEFAULT '{} '::jsonb,
  "created_at" timestamp DEFAULT now() NOT NULL,
  CONSTRAINT "payment_status_check" CHECK (status IN ('pending',
'completed', 'failed', 'refunded'))
);
```

Purpose: Tracks all payment transactions for auditing and reconciliation.

[!Note]

- Records both successful and failed payment attempts.
- `provider_transaction_id` links to Stripe/PayPal transaction IDs.
- `metadata` stores additional provider-specific information (JSON).
- Enables payment history and dispute resolution.

B. Enums Types

Enums are used to define a set of named values. They ensure data integrity by restricting the values that can be inserted into certain columns.

Key Enums and Their Purpose

1. User Role Enum

```
-- User Role Enum: Defines the roles a user can have in the system.
CREATE TYPE "public"."user_role" AS ENUM('admin', 'driver', 'customer');--
> statement-breakpoint
```

Purpose: Defines user role for role-based access control.

2. Ride Status Enum

```
-- Ride Status Enum: Defines the possible statuses of a ride.  
CREATE TYPE "public"."ride_status" AS ENUM('pending', 'assigned',  
'completed', 'cancelled');--> statement-breakpoint
```

Purpose: Defines the possible statuses of a ride, supporting ride lifecycle management.

3. Request Status Enum

```
-- Request Status Enum: Defines the possible statuses of an assignment  
request.  
CREATE TYPE "public"."request_status" AS ENUM('pending', 'approved',  
'rejected');--> statement-breakpoint
```

Purpose: Defines the possible statuses of an assignment request, supporting the assignment workflow.

4. Invoice Status Enum

```
-- Invoice Status Enum: Defines the possible statuses of an invoice.  
CREATE TYPE "public"."invoice_status" AS ENUM('unpaid', 'paid',  
'cancelled');--> statement-breakpoint
```

Purpose: Defines the possible statuses of an invoice, supporting billing and payment tracking.

5. Shift Status Enum

```
-- Shift Status Enum: Defines the possible statuses of a driver's shift.  
CREATE TYPE "public"."shift_status" AS ENUM('planned', 'active',  
'completed', 'cancelled');--> statement-breakpoint
```

Purpose: Defines the possible statuses of a driver's shift, supporting shift management.

6. Option Type Enum

```
-- Option Type Enum: Defines the possible options that can be selected for
a ride.
CREATE TYPE "public"."option_type" AS ENUM('vip', 'baby_seat', 'van',
'luxury', 'pet_friendly', 'extra_luggage', 'wheelchair_accessible',
'motorbike');--> statement-breakpoint
```

Purpose: Defines the possible options that can be selected for a ride, supporting ride customization.

7. Vehicle Type Enum

```
-- Vehicle Type Enum: Defines the possible types of vehicles a driver can
have.
CREATE TYPE "public"."vehicle_type" AS ENUM('sedan', 'suv', 'van',
'motorbike', 'luxury');--> statement-breakpoint
```

Purpose: Defines the possible types of vehicles a driver can have, supporting vehicle management.

C. Indexes and Constraints

Single-Column Indexes

```
-- Users table indexes
CREATE INDEX idx_users_role ON users(role);
CREATE INDEX idx_users_available ON users(available) WHERE role =
'driver';
CREATE INDEX idx_users_email ON users(email);

-- Geospatial index for driver location queries
CREATE INDEX idx_users_location ON users USING GIST (
    ST_MakePoint(longitude, latitude)::geography
) WHERE role = 'driver' AND available = true;

-- Rides table indexes
CREATE INDEX idx_rides_status ON rides(status);
CREATE INDEX idx_rides_departure_time ON rides(departure_time);
CREATE INDEX idx_rides_driver_id ON rides(driver_id);
CREATE INDEX idx_rides_created_at ON rides(created_at DESC);

-- Invoices table indexes
CREATE INDEX idx_invoices_status ON invoices(status);
CREATE INDEX idx_invoices_due_date ON invoices(due_date);
CREATE INDEX idx_invoices_ride_id ON invoices(ride_id);

-- Notifications table indexes
CREATE INDEX idx_notifications_user_id ON notifications(user_id);
CREATE INDEX idx_notifications_is_read ON notifications(is_read);
```

```
CREATE INDEX idx_notifications_created_at ON notifications(created_at
DESC);

-- Messages table indexes
CREATE INDEX idx_messages_sender_id ON messages(sender_id);
CREATE INDEX idx_messages_recipient_id ON messages(recipient_id);
CREATE INDEX idx_messages_ride_id ON messages(ride_id);
CREATE INDEX idx_messages_created_at ON messages(created_at DESC);

-- Activity logs index
CREATE INDEX idx_activity_logs_user_id ON activity_logs(user_id);
CREATE INDEX idx_activity_logs_created_at ON activity_logs(created_at
DESC);
```

Composite Indexes

```
-- Optimize queries for driver's active rides
CREATE INDEX idx_rides_driver_status ON rides(driver_id, status);

-- Optimize invoice reminder queries
CREATE INDEX idx_invoices_status_due_date ON invoices(status, due_date)
WHERE status = 'unpaid';

-- Optimize notification queries for unread messages
CREATE INDEX idx_notifications_user_unread ON notifications(user_id,
is_read, created_at DESC);

-- Optimize assignment request queries
CREATE INDEX idx_assignment_requests_ride_status ON
assignment_requests(ride_id, status);
CREATE INDEX idx_assignment_requests_driver_status ON
assignment_requests(driver_id, status);

-- Optimize ride customer lookups
CREATE INDEX idx_ride_customers_customer_id ON ride_customers(customer_id,
ride_id);

-- Optimize shift queries by driver and date
CREATE INDEX idx_shift_planning_driver_time ON shiftPlanning(driver_id,
start_time, end_time);
```

Summary Table

Table	Index/Constraint	Purpose
users	idx_users_available	Speed up queries for available drivers.
users	idx_users_location (GIST)	Enable fast geospatial queries (ST_DWithin).

Table	Index/Constraint	Purpose
rides	idx_rides_status	Optimize ride status filtering.
rides	idx_rides_driver_status	Fast lookup of driver's active rides.
rides	idx_rides_departure_time	Improve ride scheduling queries.
invoices	idx_invoices_status_due_date	Optimize reminder queries for unpaid bills.
notifications	idx_notifications_user_unread	Fast unread notification lookups.
messages	idx_messages_created_at	Efficient message history retrieval.
assignment_requests	idx_assignment_requests_ride_status	Optimize ride assignment workflows.
shiftPlanning	idx_shift_planning_driver_time	Fast shift schedule queries.

[!Note]

- Partial indexes (e.g., `WHERE role = 'driver'`) reduce index size and improve performance.
- GIST index on location enables fast radius searches using PostGIS.
- Composite indexes are ordered by cardinality (most selective first).
- Indexes are automatically used by Drizzle ORM's query optimizer.

D. Explanations

Tables

Each table in the database is designed to handle a specific aspect of the application, ensuring that data is organized, accessible, and secure. The relationships between tables allow for complex queries and operations, supporting the full range of functionalities required by AMT Horizon.

Enums

Enums provide a way to enforce data consistency and integrity by restricting the values that can be stored in certain columns. This ensures that only valid and expected values are used throughout the application.

Indexes and Constraints

Indexes and constraints are essential database features that ensure both **performance** and **data integrity** in AMT Horizon.

Why Indexes Are Useful

Indexes dramatically improve query performance by creating optimized data structures that allow the database to find rows faster without scanning entire tables.

Performance Benefits:

1. **Faster Data Retrieval**

- Without index: Database scans every row ($O(n)$ complexity)
- With index: Database uses B-tree or hash lookup ($O(\log n)$ complexity)
- Example: Finding available drivers goes from scanning 10,000 users to checking ~13 index entries

2. Optimized Query Plans

- PostgreSQL query planner automatically uses indexes when beneficial
- Composite indexes enable multi-column filtering without additional lookups
- Partial indexes reduce index size by only indexing relevant rows

3. Reduced Database Load

- Faster queries mean less CPU and memory usage
- Lower lock contention on frequently accessed tables
- Better concurrent user support

Real-World Impact in AMT Horizon:

```
// Without idx_rides_driver_status index:
// Scans ALL rides to find driver's active rides (slow with 100k+ rides)

// With idx_rides_driver_status index:
// Uses index to find only relevant rides instantly
const activeRides = await db.query.rides.findMany({
  where: (rides, { eq, and }) => and(
    eq(rides.driverId, driverId), // Uses index
    eq(rides.status, 'assigned')  // Uses same index
  ),
});
// Query time: 500ms → 5ms (100x faster)
```

Special Index Types:

- **GIST (Generalized Search Tree):** Enables geospatial queries for finding nearby drivers within radius
- **Partial Indexes:** Only index rows matching a condition (e.g., `WHERE role = 'driver'`), reducing index size by 66%
- **Composite Indexes:** Support queries filtering on multiple columns simultaneously

Why Constraints Are Useful

Constraints enforce data integrity rules at the database level, preventing invalid data from ever being stored.

Data Integrity Benefits:

1. Foreign Key Constraints

```
"driver_id" text REFERENCES users(id) ON DELETE SET NULL
```


- **Prevents orphaned records:** Can't assign ride to non-existent driver
- **Cascading actions:** Automatically handle deletions (SET NULL, CASCADE, RESTRICT)
- **Referential integrity:** Maintains relationships between tables

2. CHECK Constraints

```
CHECK (rating >= 1 AND rating <= 5)
CHECK (end_time > start_time)
CHECK (sender_id != recipient_id)
```

- **Business rule enforcement:** Ratings must be 1-5 stars
- **Logical validation:** End time must be after start time
- **Data quality:** Messages must be between different users

3. UNIQUE Constraints

```
CONSTRAINT "ride_customers_unique" UNIQUE("ride_id", "customer_id")
CONSTRAINT "users_email_unique" UNIQUE("email")
```

- **Prevent duplicates:** Same customer can't book same ride twice
- **Data consistency:** Email addresses must be unique across users
- **Composite uniqueness:** Enforce unique combinations of columns

4. NOT NULL Constraints

- **Required fields:** Ensure critical data is always present
- **Application reliability:** Prevents null pointer errors in code

Real-World Impact in AMT Horizon:

```
// Constraint prevents this invalid data:
await db.insert(ratings).values({
  rideId: 123,
  customerId: 'user-1',
  rating: 10, // ❌ CHECK constraint fails: rating must be 1-5
});

// Constraint prevents duplicate entries:
await db.insert(rideCustomers).values({
  rideId: 123,
  customerId: 'user-1', // ❌ UNIQUE constraint fails: already exists
});

// Foreign key maintains integrity:
await db.delete(users).where(eq(users.id, 'driver-1'));
```

```
// ✅ Automatically sets driver_id to NULL in all rides (ON DELETE SET NULL)
```

Constraints vs. Application Validation:

Aspect	Application Validation (Zod)	Database Constraints
When	Before database call	At database insertion
Bypass Risk	Can be bypassed by direct DB access	Always enforced
Error Handling	User-friendly messages	Technical database errors
Performance	Prevents invalid DB calls	Validates during write
Best Practice	Use both together	Defense in depth

Combined Approach:

```
// 1. Zod validates input (fast, user-friendly)
const CreateRatingSchema = z.object({
  rating: z.number().min(1).max(5, "Rating must be 1-5 stars"),
});

// 2. Database constraint enforces (bulletproof, always active)
CREATE TABLE ratings (
  rating integer NOT NULL CHECK (rating >= 1 AND rating <= 5)
);
```

Performance Considerations

Index Trade-offs:

- ✅ **Pros:** Faster reads, optimized queries, better user experience
- ⚠️ **Cons:** Slower writes (index must be updated), storage overhead
- 💡 **Solution:** Only index frequently queried columns

When to Use Indexes:

- ✅ Foreign key columns (driver_id, ride_id, user_id)
- ✅ Columns used in WHERE clauses (status, created_at)
- ✅ Columns used in JOIN operations
- ✅ Columns used in ORDER BY
- ❌ Low-cardinality columns (boolean with 50/50 distribution)
- ❌ Columns rarely queried
- ❌ Very small tables (< 1000 rows)

Maintenance:

```
-- Monitor index usage
SELECT schemaname, tablename, indexname, idx_scan
FROM pg_stat_user_indexes
WHERE idx_scan = 0; -- Unused indexes

-- Rebuild fragmented indexes
REINDEX INDEX idx_users_location;

-- Update statistics for query planner
ANALYZE users;
```

[!Note]

- Indexes improve **read performance** but slightly slow **write operations**
- Constraints provide **last line of defense** against data corruption
- Use **both Zod and database constraints** for comprehensive data validation
- Monitor index usage to remove unused indexes and reduce overhead
- Partial indexes are particularly effective for role-based queries in multi-tenant systems

IV. SQL Query Examples

Here are some practical SQL query examples that demonstrate how to interact with the database:

A. List unassigned rides

```
SELECT * FROM rides
WHERE status = 'pending' AND driver_id IS NULL;
```

Purpose: Retrieve all rides not yet assigned to a driver.

B. Generate an invoice

```
INSERT INTO invoices (ride_id, amount, status, invoice_date, due_date)
VALUES (1, 25.50, 'unpaid', NOW(), NOW() + INTERVAL '30 days');
```

Purpose: Create a new invoice for a ride, setting the due date to 30 days from now.

C. Update assignment request status

```
UPDATE assignment_requests
SET status = 'approved'
```

```
WHERE request_id = 1;
```

Purpose: Approve a driver's request to be assigned to a ride.

D. Retrieve user's unread notifications

```
SELECT * FROM notifications
WHERE user_id = 1 AND is_read = FALSE;
```

Purpose: Fetch all unread notifications for a specific user.

E. List available drivers within a 5 km radius

```
-- Assuming `latitude` and `longitude` columns in the users table
SELECT u.user_id, u.email
FROM users u
WHERE u.role = 'driver'
      AND u.available = TRUE
      AND ST_DWithin(
        ST_MakePoint(u.longitude, u.latitude)::geography,
        ST_MakePoint(:client_longitude, :client_latitude)::geography,
        5000 -- 5 km in meters
      );
```

Purpose: Find all available drivers within a 5 km radius of a specified location.

F. Calculate driver's average rating

```
SELECT AVG(rating) as average_rating
FROM ratings
WHERE driver_id = :driver_id;
```

Purpose: Calculate the average rating for a specific driver based on customer feedback.

G. Send invoice via application

```
UPDATE invoices
SET
    sent_via_app = TRUE,
```

```
    app_sent_at = NOW(),
    status = 'sent'
WHERE invoice_id = :invoice_id;
-- Send PDF by email and in-app notification (backend logic)
```

Purpose: Mark an invoice as sent via the application and update its status.

H. List invoices eligible for reminder (unpaid, overdue)

```
SELECT invoice_id, ride_id, amount, due_date
FROM invoices
WHERE status = 'unpaid'
      AND due_date < NOW()
      AND reminders_sent < 3;
```

Purpose: Identify all unpaid and overdue invoices that are eligible for a reminder.

I. Send manual reminder

```
UPDATE invoices
SET
    reminders_sent = reminders_sent + 1,
    last_reminder_message = :custom_message,
    status = 'reminder_sent'
WHERE invoice_id = :invoice_id;
-- Send email/notification with custom message
```

Purpose: Send a manual reminder for an overdue invoice and update its status.

J. Drizzle ORM Query Examples

Here are the Drizzle ORM equivalents for common queries:

List unassigned rides

```
import { db } from '@lib/db';
import { rides } from '@lib/db/schema';
import { eq, isNull } from 'drizzle-orm';

const unassignedRides = await db.query.rides.findMany({
  where: (rides, { eq, isNull, and }) => and(
    eq(rides.status, 'pending'),
    isNull(rides.driverId)
  )
});
```

```
),  
});
```

Find available drivers within 5 km radius

```
import { db } from '@lib/db';  
import { users } from '@lib/db/schema';  
import { eq, sql } from 'drizzle-orm';  
  
const nearbyDrivers = await db  
  .select({  
    id: users.id,  
    name: users.name,  
    email: users.email,  
    vehicleType: users.vehicleType,  
  })  
  .from(users)  
  .where(  
    sql`${users.role} = 'driver'  
      AND ${users.available} = true  
      AND ST_DWithin(  
        ST_MakePoint(${users.longitude}, ${users.latitude}>::geography,  
        ST_MakePoint(${clientLongitude}, ${clientLatitude}>::geography,  
        5000  
      )`  
  );
```

Retrieve user's unread notifications

```
import { db } from '@lib/db';  
import { notifications } from '@lib/db/schema';  
import { eq, and } from 'drizzle-orm';  
  
const unreadNotifications = await db.query.notifications.findMany({  
  where: (notifications, { eq, and }) => and(  
    eq(notifications.userId, userId),  
    eq(notifications.isRead, false)  
  ),  
  orderBy: (notifications, { desc }) => [desc(notifications.createdAt)],  
});
```

Calculate driver's average rating

```
import { db } from '@lib/db';  
import { ratings } from '@lib/db/schema';  
import { eq, avg } from 'drizzle-orm';
```

```
const driverRating = await db
  .select({
    averageRating: avg(ratings.rating),
  })
  .from(ratings)
  .where(eq(ratings.driverId, driverId));
```

Get ride with full details (including customers and driver)

```
import { db } from '@lib/db';

const rideDetails = await db.query.rides.findFirst({
  where: (rides, { eq }) => eq(rides.id, rideId),
  with: {
    driver: true,
    rideCustomers: {
      with: {
        customer: true,
      },
    },
    selectedOptions: true,
    invoices: true,
    ratings: {
      with: {
        customer: true,
      },
    },
  },
});
```

List invoices eligible for reminder

```
import { db } from '@lib/db';
import { invoices } from '@lib/db/schema';
import { eq, and, lt } from 'drizzle-orm';

const overdueInvoices = await db.query.invoices.findMany({
  where: (invoices, { eq, and, lt }) => and(
    eq(invoices.status, 'unpaid'),
    lt(invoices.dueDate, new Date()),
    lt(invoices.remindersSent, 3)
  ),
  with: {
    ride: {
      with: {
        rideCustomers: {
          with: {
            customer: true,
          },
        },
      },
    },
  },
});
```

```

    },
  },
},
},
});

```

[!Note]

- Drizzle provides type-safe queries with auto-completion
- Use `db.query` for relational queries with `with` for joins
- Use `db.select()` for more complex queries with custom SQL
- All queries are automatically parameterized to prevent SQL injection

K. Database Migration Strategy

Overview

AMT Horizon uses **Drizzle Kit** for database schema management and migrations. This ensures version-controlled, reproducible database changes across all environments.

Migration Workflow

```

flowchart LR
    A[Modify schema.ts] --> B[Generate migration]
    B --> C[Review migration SQL]
    C --> D{Approved?}
    D -- Yes --> E[Apply to dev]
    E --> F[Test thoroughly]
    F --> G[Apply to staging]
    G --> H[Final validation]
    H --> I[Apply to production]
    D -- No --> A

```

Step-by-Step Process

1. Generate Migration

```

# After modifying lib/db/schema.ts
pnpm drizzle-kit generate:pg

```

This creates a new migration file in `drizzle/` directory:

```

drizzle/
├── 0000_initial_schema.sql

```



```
├─ 0001_add_location_fields.sql
├─ meta/
│   └─ _journal.json
│   └─ 0001_snapshot.json
```

2. Review Migration SQL

Always review generated SQL before applying:

```
-- Example migration file: 0001_add_location_fields.sql
ALTER TABLE "users" ADD COLUMN "latitude" numeric(10, 7);
ALTER TABLE "users" ADD COLUMN "longitude" numeric(10, 7);
ALTER TABLE "users" ADD COLUMN "last_location_update" timestamp;

CREATE INDEX "idx_users_location" ON "users" USING GIST (
  ST_MakePoint(longitude, latitude)::geography
) WHERE role = 'driver' AND available = true;
```

3. Apply Migration

Development:

```
# Push directly to dev database (no migration files needed)
pnpm drizzle-kit push:pg
```

Staging/Production:

```
# Apply migration files
pnpm drizzle-kit migrate
```

4. Rollback Strategy

Create backup before migration:

```
# Backup production database
pg_dump $DATABASE_URL > backups/prod_backup_$(date +%Y%m%d_%H%M%S).sql
```

Manual rollback if needed:

```
-- Create down migration manually
-- Example: rollback_0001_add_location_fields.sql
DROP INDEX IF EXISTS "idx_users_location";
```

```
ALTER TABLE "users" DROP COLUMN "last_location_update";
ALTER TABLE "users" DROP COLUMN "longitude";
ALTER TABLE "users" DROP COLUMN "latitude";
```

Migration Best Practices

1. Always Test First

- Apply migrations to local/dev environment first
- Run full test suite after migration
- Verify data integrity

2. Backwards Compatibility

- Add columns as nullable first, then backfill data
- Use separate migrations for:
 1. Add column (nullable)
 2. Backfill data
 3. Add NOT NULL constraint

3. Breaking Changes

```
// Bad: Dropping column immediately
// ALTER TABLE users DROP COLUMN old_field;

// Good: Deprecate first, drop later
// 1. Add new column
// 2. Backfill data
// 3. Update code to use new column
// 4. Deploy
// 5. Drop old column in next migration
```

4. Large Data Migrations

```
-- For tables with millions of rows, use batching
UPDATE rides
SET normalized_address = normalize(departure)
WHERE id IN (
  SELECT id FROM rides
  WHERE normalized_address IS NULL
  LIMIT 1000
);
-- Run multiple times until complete
```

5. Index Creation

```
-- Create indexes concurrently to avoid locking
CREATE INDEX CONCURRENTLY idx_rides_departure_time
ON rides(departure_time);
```

drizzle.config.ts

```
import type { Config } from 'drizzle-kit';

export default {
  schema: './lib/db/schema.ts',
  out: './drizzle',
  driver: 'pg',
  dbCredentials: {
    connectionString: process.env.DATABASE_URL!,
  },
  verbose: true,
  strict: true,
} satisfies Config;
```

Migration Checklist

- ☐ Schema changes documented in migration file
- ☐ Migration tested in local environment
- ☐ Database backup created
- ☐ Migration reviewed by team lead
- ☐ Applied to staging environment
- ☐ Application tested in staging
- ☐ Rollback plan documented
- ☐ Production deployment scheduled
- ☐ Migration applied to production
- ☐ Post-migration verification completed

[!Warning]

- Never modify existing migration files after they've been applied
- Always create backups before production migrations
- Test rollback procedures in staging first
- For Neon, use branching feature to test migrations safely

V. Code Actions (Server Actions and Components)

Code Actions replace traditional API endpoints. They are defined in dedicated files (e.g., `lib/actions/signActions.ts`) and called directly from server components.

A. Code Action Structure

Each action us an async function using Drizzle ORM for CRUD operations.

Example: Create a Ride

```
import { z } from "zod";
import { db } from "@/lib/db";
import { rides, rideCustomers } from "@/lib/db/schema";

const CreateRideSchema = z.object({
  departure: z.string().min(3, "Departure must be at least 3 characters"),
  destination: z.string().min(3, "Destination must be at least 3
characters"),
  departureTime: z.date().min(new Date(), "Departure time must be in the
future"),
  customerIds: z.array(z.string()).nonempty("At least one customer is
required"),
  price: z.number().positive("Price must be positive").optional(),
  driverId: z.string().optional(),
  status: z.enum(['pending', 'assigned', 'completed',
'cancelled']).optional(),
});

export async function createRide(data: unknown) {
  try {
    const validatedData = CreateRideSchema.parse(data);
    const {departureTime, customerIds, departure, destination,
driverId, price, status} = validatedData;

    const [newRide] = await db
      .insert(rides)
      .values({
        departure,
        destination,
        departureTime,
        price: price || 0,
        status: status || 'pending',
        driverId: driverId || null,
      })
      .returning({id: rides.id});

    if (customerIds.length > 0) {
      await db.insert(rideCustomers).values(
        customerIds.map(customerId => ({
          rideId: newRide.id,
          customerId,
        })))
    };
  }

  return { success: true, data: newRide.id };
} catch (error) {
  if (error instanceof z.ZodError) {
```

```
        return { success: false, error: 'Validation failed', details:
error.flatten() };
    }
    console.error('Error creating ride:', error);
    return { success: false, error: 'Failed to create ride' };
  }
}
```

[!Note]

- Zod validation catches invalid inputs before database operations.
- Returns structured response with `success` flag for consistent error handling.
- Validation errors include detailed field-level information.

B. Main Code Actions

Action	Description	File
<code>createRide</code>	Create a new ride.	<code>app/actions/ride.ts</code>
<code>assignDriverToRide</code>	Assign a driver to a ride.	<code>app/actions/assignment.ts</code>
<code>generateInvoice</code>	Generate an invoice for a ride.	<code>app/actions/invoice.ts</code>
<code>sendInvoiceReminder</code>	Send a reminder for an unpaid invoice.	<code>app/actions/invoice.ts</code>
<code>updateRideStatus</code>	Update ride status (e.g., "completed").	<code>app/actions/ride.ts</code>
<code>requestAssignment</code>	Driver requests assignment to a ride.	<code>app/actions/assignment.ts</code>
<code>calculateDriverRating</code>	Calculate a driver's average rating.	<code>app/actions/rating.ts</code>
<code>importRideFromPhoto</code>	Import a ride from a photo (OCR).	<code>app/actions/ocr.ts</code>
<code>listUnassignedRides</code>	List unassigned rides.	<code>app/actions/ride.ts</code>
<code>getDriverShifts</code>	Retrieve a driver's shift schedule.	<code>app/actions/shift.ts</code>

C. API Response Format

All Server Actions return a standardized response format for consistent error handling:

```
// Standard response type
type ActionResponse<T> =
  | { success: true; data: T }
  | { success: false; error: string; code?: string; details?: unknown };

// Usage example
export async function assignDriverToRide(
  rideId: number,
  driverId: string
): Promise<ActionResponse<void>> {
```

```
try {
  const ride = await db.query.rides.findFirst({
    where: (rides, { eq }) => eq(rides.id, rideId),
  });

  if (!ride) {
    return { success: false, error: 'Ride not found', code:
'RIDE_NOT_FOUND' };
  }

  if (ride.status !== 'pending') {
    return {
      success: false,
      error: 'Ride is not available for assignment',
      code: 'RIDE_NOT_AVAILABLE'
    };
  }

  await db
    .update(rides)
    .set({ driverId, status: 'assigned' })
    .where(eq(rides.id, rideId));

  return { success: true, data: undefined };
} catch (error) {
  console.error('Error assigning driver:', error);
  return { success: false, error: 'Failed to assign driver', code:
'ASSIGNMENT_ERROR' };
}
```

[!Note]

- Use error codes for programmatic error handling on the frontend.
- Include `details` field for validation errors (Zod errors).
- Log errors server-side but return user-friendly messages.

D. Validation Schemas

All Server Actions use Zod schemas for input validation:

```
// lib/validations/ride.ts
import { z } from 'zod';

export const CreateRideSchema = z.object({
  departure: z.string().min(3, "Departure must be at least 3 characters"),
  destination: z.string().min(3, "Destination must be at least 3
characters"),
  departureTime: z.date().min(new Date(), "Departure time must be in the
future"),
});
```

```
    customerIds: z.array(z.string()).nonempty("At least one customer is required"),
    price: z.number().positive("Price must be positive").optional(),
    driverId: z.string().optional(),
  });

export const AssignDriverSchema = z.object({
  rideId: z.number().int().positive(),
  driverId: z.string().min(1, "Driver ID is required"),
});

export const UpdateRideStatusSchema = z.object({
  rideId: z.number().int().positive(),
  status: z.enum(['pending', 'assigned', 'completed', 'cancelled']),
  notes: z.string().optional(),
});

// lib/validations/invoice.ts
export const GenerateInvoiceSchema = z.object({
  rideId: z.number().int().positive(),
  discountPercent: z.number().min(0).max(100).optional(),
  discountReason: z.string().optional(),
});

export const ProcessPaymentSchema = z.object({
  invoiceId: z.number().int().positive(),
  amount: z.number().positive(),
  paymentMethod: z.enum(['card', 'cash', 'bank_transfer']),
  currency: z.string().length(3).default('EUR'),
});

// lib/validations/user.ts
export const UpdateUserSchema = z.object({
  userId: z.string(),
  name: z.string().min(2).optional(),
  email: z.string().email().optional(),
  phone: z.string().regex(/^\+?[1-9]\d{1,14}$/).optional(),
  vehicleType: z.enum(['sedan', 'suv', 'van', 'motorbike', 'luxury']).optional(),
});
```

[!Note]

- Centralize validation schemas in `lib/validations/` for reusability.
- Use descriptive error messages for better UX.
- Schemas can be reused for both server and client-side validation.

E. Error Handling

```
export async function assignDriverToRide(rideId: number, driverId: string)
{
  const ride = await db.query.rides.findFirst({
    where: (rides, { eq }) => eq(rides.id, rideId),
  });

  if (!ride) throw new Error('Ride not found');
  if (ride.status !== 'pending') throw new Error('Ride is not available
for assignment');

  try {
    await db
      .update(rides)
      .set({
        driverId,
        status: 'assigned'
      })
      .where(eq(rides.id, rideId));
  } catch (error) {
    console.error('Error assigning driver:', error);
    throw new Error('Failed to assign driver');
  }
}
```

F. Transactions

```
export async function createRideWithInvoice(data: unknown) {
  const validatedData = CreateRideSchema.parse(data);

  return await db.transaction(async (tx) => {
    const [newRide] = await tx
      .insert(rides)
      .values({ ...validatedData, status: "pending" })
      .returning({ id: rides.id });

    await tx.insert(invoices).values({
      rideId: newRide.id,
      subTotal: validatedData.price || 0,
      total: validatedData.price || 0,
    });

    return newRide.id;
  });
}
```

Additional Transaction Examples


```
// Example: Process payment with transaction
export async function processPayment(invoiceId: number, amount: number) {
  return await db.transaction(async (tx) => {
    // 1. Verify invoice exists and is unpaid
    const invoice = await tx.query.invoices.findFirst({
      where: (invoices, { eq }) => eq(invoices.id, invoiceId),
    });

    if (!invoice || invoice.status === 'paid') {
      throw new Error('Invoice not found or already paid');
    }

    // 2. Record payment transaction
    await tx.insert(paymentTransactions).values({
      invoiceId,
      amount,
      paymentMethod: 'card',
      provider: 'stripe',
      status: 'completed',
    });

    // 3. Update invoice
    const newPaidAmount = Number(invoice.paidAmount) + amount;
    const newStatus = newPaidAmount >= Number(invoice.total) ? 'paid' :
    'unpaid';

    await tx
      .update(invoices)
      .set({
        paidAmount: newPaidAmount.toString(),
        status: newStatus,
      })
      .where(eq(invoices.id, invoiceId));

    return { success: true, newStatus };
  });
}

// Example: Assign driver with conflict resolution
export async function assignDriverWithLock(rideId: number, driverId:
string) {
  return await db.transaction(async (tx) => {
    // Lock the ride row to prevent race conditions
    const ride = await tx.query.rides.findFirst({
      where: (rides, { eq }) => eq(rides.id, rideId),
      // FOR UPDATE lock in raw SQL if needed:
      // extras: { forUpdate: true }
    });

    if (!ride || ride.status !== 'pending') {
      throw new Error('Ride not available');
    }
  });
}
```

```
// Update ride
await tx
  .update(rides)
  .set({ driverId, status: 'assigned' })
  .where(eq(rides.id, rideId));

// Reject other pending requests for this ride
await tx
  .update(assignmentRequests)
  .set({ status: 'rejected' })
  .where(
    and(
      eq(assignmentRequests.rideId, rideId),
      eq(assignmentRequests.status, 'pending')
    )
  );

return { success: true };
});
}
```

[!Note]

- Use transactions for operations that must succeed or fail together.
- Transactions prevent partial updates and race conditions.
- All operations within a transaction are automatically rolled back on error.

VI. Business Logic

A. Key Algorithms

1. Automatic Driver Assignment

```
flowchart LR
  A[Start] --> B["Fetch available drivers within 5 km"]
  B --> C["Filter by vehicle type and rating"]
  C --> D["Sort by distance and rating"]
  D --> E["Assign closest and highest-rated driver"]
  E --> F["Update ride and assignment status"]
```

[!Note]

- The system uses geolocation queries to find drivers within a 5 km radius.
- Priority is given to drivers with higher ratings and closer proximity.
- If no drivers are available, the ride remains in **pending** status.
- **Code Action:** `assignDriverToRide` (called manually or automatically).

2. Price Calculation

flowchart LR

```
A[Price Calculation] --> B["Calculate base price (fixed + distance)"]
B --> C["Add selected options' prices"]
C --> D["Include extra fees (waiting time, peak hours)"]
D --> E[Return total price]
```

[!Note]

- Base price includes a fixed fee plus distance-based charges (€1.50 per km).
 - Waiting time is charged at €0.50 per minute.
 - Optional services (VIP, baby seat, etc.) add to the total price.
- **Code Action:** `calculateRidePrice` (called before ride creation).

3. Invoice Management

- **Set Invoice Status**

flowchart LR

```
A["When created, set to 'unpaid'"]
A --> B{Is payment received?}
B -- Yes --> C["Set to 'paid'"]
B -- No --> D{Is payment cancelled?}
D -- Yes --> E["Set status to 'cancelled'"]
D -- No --> F{"Is 14 days since invoice creation or 7 days since last reminder?"}
F -- Yes --> G[Send reminder]
F -- No --> H[Wait]
```

[!Note]

- Invoices are automatically set to `unpaid` upon creation.
 - Payment reminders are sent at D+14, D+21, and D+30 after the due date.
 - Maximum of 3 reminders per invoice to avoid spam.
- **Code Actions:**
 - Generate Invoice: `generateInvoice`.
 - Send Reminder: `sendInvoiceReminder`.

B. Business Logic in Code Actions

Example: Ride Price Calculation

```
export async function calculateRidePrice(
  basePrice: number,
  distanceKm: number,
  options: { name: string; price: number }[],
  waitingTime: number = 0
) {
  const optionsTotal = options.reduce((sum, option) => sum + option.price,
0);
  const waitingFee = waitingTime * 0.5; // €0.50 per minute
  const total = basePrice + distanceKm * 1.5 + optionsTotal + waitingFee;

  return { total, breakdown: { basePrice, distanceKm, optionsTotal,
waitingFee } };
}
```

[!Note]

- This function provides a detailed breakdown of all charges for transparency.
- The price is locked at booking time to avoid surprises for customers.

VII. Authentication & Security

A. Authentication

- **better-auth**: Manages sessions and roles.
- **Code Action Protection**:
 - Use `auth()` to check permissions before execution.
 - Example:

```
import { auth } from "@auth";

export async function adminOnlyAction() {
  const session = await auth();
  if (session?.user.role !== "admin") {
    throw new Error("Unauthorized");
  }
}
```

[!Note]

- All sensitive actions require role verification before execution.
- Two-factor authentication is mandatory for admins and optional for other users.

B. Data Security

- **Zod:** Validates inputs.
- **Drizzle:** Sanitizes queries.
- **better-auth:** Secures sessions.

[!Note]

- All user data is encrypted at rest and in transit.
- Password reset links are valid for 24 hours only.
- Session tokens expire after a period of inactivity.

VIII. External Integrations

A. Geolocation (Mapbox/Google Maps)

- **Usage:**
 - Calculate distance between points.
 - Find available drivers within a radius.
- **Code Action:** `findNearbyDrivers` (uses `ST_DWithin` via Drizzle).

[!Note]

- Geolocation services provide real-time tracking for drivers and customers.
- Distance calculations use the Haversine formula for accuracy.

B. Payments (Stripe/PayPal)

- **Flow:**

```
flowchart LR
    A[User Initiates Payment] --> B[Generate Invoice]
    B --> C[Redirect to Stripe/PayPal]
    C --> D[Payment Confirmation]
    D --> E[Update Invoice Status]
```

[!Note]

- Partial payments are allowed (minimum 30% of the total amount).
- Payment confirmations trigger automatic status updates and notifications.

- **Code Action:** `processPayment`.

C. OCR (Tesseract.js)

- **Usage:**
 - Extract text from ride/ticket photos.
 - Automatically create a ride.

- **Code Action:** `importRideFromPhoto`.

[!Note]

- OCR accuracy must be at least 90% for automatic import.
- If OCR fails or data is unclear, users can manually correct the extracted information.
- Supported formats include photos of tickets, invoices, and handwritten notes.

D. Rate Limiting

Strategy: Prevent abuse of Server Actions using middleware or library (e.g., `next-rate-limit`).

```
// lib/rateLimit.ts
import rateLimit from 'express-rate-limit';

const limiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 minutes
  max: 100, // Limit each IP to 100 requests per windowMs
  message: 'Too many requests, please try again later.',
});

export const rateLimitConfig = {
  createRide: { max: 20, windowMs: 60 * 1000 }, // 20 rides per minute
  sendMessage: { max: 50, windowMs: 60 * 1000 }, // 50 messages per minute
  processPayment: { max: 5, windowMs: 60 * 1000 }, // 5 payments per
minute
};
```

[!Note]

- Different limits for different action types based on sensitivity.
- Can use Redis for distributed rate limiting in production.
- Rate limiting helps prevent DoS attacks and spam.

E. Caching Strategy

Frequently Accessed Data: Cache static or slowly-changing data to reduce database load.

```
// Example: Cache ride options
import { cache } from 'react';

export const getRideOptions = cache(async () => {
  return await db.query.rideOptions.findMany();
});

// Example: Cache driver ratings (with 5-minute TTL)
import { unstable_cache } from 'next/cache';
```

```
export const getDriverRating = unstable_cache(
  async (driverId: string) => {
    const ratings = await db.query.ratings.findMany({
      where: (ratings, { eq }) => eq(ratings.driverId, driverId),
    });

    if (ratings.length === 0) return null;

    const average = ratings.reduce((sum, r) => sum + r.rating, 0) /
ratings.length;
    return { average, count: ratings.length };
  },
  ['driver-rating'],
  { revalidate: 300, tags: ['ratings'] } // 5-minute cache
);
```

[!Note]

- Use Next.js built-in caching for better performance.
- Cache invalidation can be triggered with `revalidateTag()`.
- For high-traffic applications, consider Redis for distributed caching.

F. File Storage & OCR

Purpose: Handle photo uploads for OCR processing (ride tickets, invoices).

Storage Options:

- **Vercel Blob:** For Next.js projects on Vercel
- **AWS S3:** For enterprise deployments
- **Cloudinary:** For image optimization

```
// Example: Upload and process ride photo
import { put } from '@vercel/blob';
import Tesseract from 'tesseract.js';

export async function importRideFromPhoto(file: File) {
  try {
    // 1. Upload to blob storage
    const blob = await put(`rides/${Date.now()}-${file.name}`, file, {
      access: 'public',
    });

    // 2. Extract text via OCR
    const { data: { text } } = await Tesseract.recognize(blob.url, 'eng');

    // 3. Parse extracted data
    const rideData = parseRideFromText(text);

    // 4. Validate and create ride
```

```

    const validatedData = CreateRideSchema.parse(rideData);

    const rideId = await createRide({
      ...validatedData,
      photoUrl: blob.url,
    });

    return { success: true, data: { rideId, photoUrl: blob.url, accuracy:
0.95 } };
  } catch (error) {
    if (error instanceof z.ZodError) {
      return {
        success: false,
        error: 'OCR data validation failed',
        details: error.flatten(),
        requiresManualInput: true
      };
    }
    return { success: false, error: 'OCR processing failed' };
  }
}

function parseRideFromText(text: string) {
  // Custom parsing logic based on ticket format
  // Extract: departure, destination, time, price, etc.
  // Return structured data
}

```

[!Note]

- OCR accuracy must be $\geq 90\%$ for automatic import (per requirements).
- Failed OCR prompts manual correction UI.
- Store original photo for audit trail.

G. Real-time Communication

Purpose: Enable real-time updates for ride tracking, notifications, and chat.

Technology: Server-Sent Events (SSE) or WebSockets via Next.js Route Handlers.

```

// app/api/sse/ride-updates/route.ts
export async function GET(request: Request) {
  const { searchParams } = new URL(request.url);
  const rideId = searchParams.get('rideId');

  const encoder = new TextEncoder();
  const stream = new ReadableStream({
    async start(controller) {
      // Send initial data
      const ride = await db.query.rides.findFirst({

```



```

    where: (rides, { eq }) => eq(rides.id, Number(rideId)),
  });

  controller.enqueue(
    encoder.encode(`data: ${JSON.stringify(ride)}\n\n`)
  );

  // Poll for updates every 5 seconds
  const interval = setInterval(async () => {
    const updatedRide = await db.query.rides.findFirst({
      where: (rides, { eq }) => eq(rides.id, Number(rideId)),
    });

    controller.enqueue(
      encoder.encode(`data: ${JSON.stringify(updatedRide)}\n\n`)
    );
  }, 5000);

  // Cleanup on connection close
  request.signal.addEventListener('abort', () => {
    clearInterval(interval);
    controller.close();
  });
},
});

return new Response(stream, {
  headers: {
    'Content-Type': 'text/event-stream',
    'Cache-Control': 'no-cache',
    'Connection': 'keep-alive',
  },
});
}

```

Client Usage:

```

// components/RideTracker.tsx
useEffect(() => {
  const eventSource = new EventSource(`/api/sse/ride-updates?rideId=${rideId}`);

  eventSource.onmessage = (event) => {
    const rideData = JSON.parse(event.data);
    setRide(rideData);
  };

  return () => eventSource.close();
}, [rideId]);

```

[!Note]

- SSE is simpler than WebSockets for one-way updates (server → client).
- For bidirectional communication (chat), use WebSocket libraries like Socket.io.
- Consider using Pusher or Ably for managed real-time infrastructure.

H. Background Jobs & Scheduled Tasks

Purpose: Automate recurring tasks such as invoice reminders, data cleanup, and reporting.

Scheduled Tasks Overview

Task	Schedule	Purpose	Implementation
Invoice Reminders	Daily at 9 AM	Send reminders for overdue invoices (D+14, D+21, D+30)	Vercel Cron
Activity Log Cleanup	Daily at 2 AM	Delete logs older than 6 months (GDPR compliance)	Vercel Cron
Message Cleanup	Daily at 3 AM	Delete messages older than 30 days	Vercel Cron
Driver Availability Check	Every 15 minutes	Update driver availability based on last location update	Vercel Cron
Monthly Reports	1st of month at 8 AM	Generate monthly performance reports	Vercel Cron
Shift Status Update	Every hour	Update shift statuses (planned → active → completed)	Vercel Cron

Implementation with Vercel Cron

Configuration: vercel.json

```
{
  "crons": [
    {
      "path": "/api/cron/invoice-reminders",
      "schedule": "0 9 * * *"
    },
    {
      "path": "/api/cron/cleanup-logs",
      "schedule": "0 2 * * *"
    },
    {
      "path": "/api/cron/cleanup-messages",
      "schedule": "0 3 * * *"
    },
    {
      "path": "/api/cron/check-driver-availability",
```

```

    "schedule": "*/15 * * * *"
  },
  {
    "path": "/api/cron/monthly-reports",
    "schedule": "0 8 1 * *"
  }
]
}

```

Example: Invoice Reminder Job

```

// app/api/cron/invoice-reminders/route.ts
import { NextRequest, NextResponse } from 'next/server';
import { db } from '@/lib/db';
import { invoices } from '@/lib/db/schema';
import { eq, and, lt } from 'drizzle-orm';
import { sendEmail } from '@/lib/email';

export async function GET(request: NextRequest) {
  // Verify cron secret to prevent unauthorized access
  const authHeader = request.headers.get('authorization');
  if (authHeader !== `Bearer ${process.env.CRON_SECRET}`) {
    return NextResponse.json({ error: 'Unauthorized' }, { status: 401 });
  }

  try {
    const now = new Date();
    const fourteenDaysAgo = new Date(now.getTime() - 14 * 24 * 60 * 60 * 1000);

    // Find invoices needing first reminder (D+14)
    const firstReminders = await db.query.invoices.findMany({
      where: (invoices, { eq, and, lt }) => and(
        eq(invoices.status, 'unpaid'),
        lt(invoices.dueDate, fourteenDaysAgo),
        eq(invoices.remindersSent, 0)
      ),
      with: {
        ride: {
          with: {
            rideCustomers: {
              with: { customer: true },
            },
          },
        },
      },
    });

    // Send first reminders
    for (const invoice of firstReminders) {
      await sendEmail({

```

```

    to: invoice.ride.rideCustomers[0].customer.email,
    subject: 'Payment Reminder – Invoice Overdue',
    template: 'invoice-reminder',
    data: {
      invoiceId: invoice.id,
      amount: invoice.total,
      dueDate: invoice.dueDate,
      reminderNumber: 1,
    },
  });

  await db
    .update(invoices)
    .set({
      remindersSent: 1,
      lastReminderMessage: 'First reminder sent',
    })
    .where(eq(invoices.id, invoice.id));
}

return NextResponse.json({
  success: true,
  remindersSent: firstReminders.length,
});
} catch (error) {
  console.error('Invoice reminder job failed:', error);
  return NextResponse.json({ error: 'Job failed' }, { status: 500 });
}
}

```

Example: Activity Log Cleanup

```

// app/api/cron/cleanup-logs/route.ts
import { NextRequest, NextResponse } from 'next/server';
import { db } from '@lib/db';
import { activityLogs } from '@lib/db/schema';
import { lt } from 'drizzle-orm';

export async function GET(request: NextRequest) {
  const authHeader = request.headers.get('authorization');
  if (authHeader !== `Bearer ${process.env.CRON_SECRET}`) {
    return NextResponse.json({ error: 'Unauthorized' }, { status: 401 });
  }

  try {
    const sixMonthsAgo = new Date();
    sixMonthsAgo.setMonth(sixMonthsAgo.getMonth() - 6);

    const result = await db
      .delete(activityLogs)
      .where(lt(activityLogs.createdAt, sixMonthsAgo));
  }
}

```

```

    return NextResponse.json({
      success: true,
      deletedCount: result.rowCount,
    });
  } catch (error) {
    console.error('Log cleanup job failed:', error);
    return NextResponse.json({ error: 'Job failed' }, { status: 500 });
  }
}

```

[!Note]

- All cron endpoints require authentication via `CRON_SECRET` environment variable
- Vercel cron jobs have a 10-second execution limit (use for triggering longer jobs)
- Monitor cron execution via activity logs and error tracking
- Test cron jobs locally using: `curl http://localhost:3000/api/cron/[job-name] -H "Authorization: Bearer [secret]"`

I. Performance Optimization

Query Optimization

1. Prevent N+1 Queries

```

// ❌ Bad: N+1 query problem
const rides = await db.query.rides.findMany();
for (const ride of rides) {
  const driver = await db.query.users.findFirst({
    where: eq(users.id, ride.driverId),
  });
}

// ✅ Good: Use joins/relations
const rides = await db.query.rides.findMany({
  with: {
    driver: true,
    rideCustomers: {
      with: { customer: true },
    },
  },
});

```

2. Use Proper Indexes

```

// Query benefits from idx_rides_driver_status
const driverRides = await db.query.rides.findMany({
  where: (rides, { eq, and }) => and(

```

```
    eq(rides.driverId, driverId),
    eq(rides.status, 'assigned')
  ),
});
```

3. Pagination for Large Datasets

```
import { desc, gt } from 'drizzle-orm';

// Cursor-based pagination (better for large datasets)
export async function getRides(cursor?: number, limit = 20) {
  const rides = await db.query.rides.findMany({
    where: cursor ? gt(rides.id, cursor) : undefined,
    limit: limit + 1, // Fetch one extra to check if there's more
    orderBy: [desc(rides.createdAt)],
  });

  const hasMore = rides.length > limit;
  const results = hasMore ? rides.slice(0, -1) : rides;
  const nextCursor = hasMore ? results[results.length - 1].id : null;

  return { rides: results, nextCursor, hasMore };
}
```

4. Select Only Needed Columns

```
// ❌ Bad: Select all columns
const users = await db.select().from(users);

// ✅ Good: Select specific columns
const drivers = await db
  .select({
    id: users.id,
    name: users.name,
    vehicleType: users.vehicleType,
    available: users.available,
  })
  .from(users)
  .where(eq(users.role, 'driver'));
```

Connection Pooling

Neon Serverless Driver Configuration:

```
// lib/db/drizzle.ts
import { neon } from '@neondatabase/serverless';
import { drizzle } from 'drizzle-orm/neon-http';
```

```
import * as schema from './schema';

const sql = neon(process.env.DATABASE_URL!, {
  fetchOptions: {
    cache: 'no-store', // Disable caching for dynamic data
  },
});

export const db = drizzle(sql, { schema });
```

Caching Strategies

1. Static Data Caching

```
import { unstable_cache } from 'next/cache';

// Cache ride options (rarely change)
export const getRideOptions = unstable_cache(
  async () => {
    return await db.query.rideOptions.findMany();
  },
  ['ride-options'],
  { revalidate: 3600, tags: ['options'] } // 1 hour cache
);
```

2. Request Deduplication

```
import { cache } from 'react';

// Deduplicate requests within a single render
export const getUser = cache(async (userId: string) => {
  return await db.query.users.findFirst({
    where: eq(users.id, userId),
  });
});
```

[!Note]

- Aim for queries under 100ms for optimal user experience
- Use Neon's query performance dashboard to identify bottlenecks
- Enable query logging in development to catch issues early

J. Webhook Handling

Stripe Webhook Integration

Purpose: Handle payment confirmations, failures, and refunds from Stripe.

```
// app/api/webhooks/stripe/route.ts
import { NextRequest, NextResponse } from 'next/server';
import Stripe from 'stripe';
import { db } from '@lib/db';
import { invoices, paymentTransactions } from '@lib/db/schema';
import { eq } from 'drizzle-orm';

const stripe = new Stripe(process.env.STRIPE_SECRET_KEY!, {
  apiVersion: '2023-10-16',
});

const webhookSecret = process.env.STRIPE_WEBHOOK_SECRET!;

export async function POST(request: NextRequest) {
  const body = await request.text();
  const signature = request.headers.get('stripe-signature')!;

  let event: Stripe.Event;

  try {
    event = stripe.webhooks.constructEvent(body, signature,
webhookSecret);
  } catch (err) {
    console.error('Webhook signature verification failed:', err);
    return NextResponse.json({ error: 'Invalid signature' }, { status: 400
});
  }

  try {
    switch (event.type) {
      case 'payment_intent.succeeded':
        await handlePaymentSuccess(event.data.object as
Stripe.PaymentIntent);
        break;

        case 'payment_intent.payment_failed':
          await handlePaymentFailure(event.data.object as
Stripe.PaymentIntent);
          break;

          case 'charge.refunded':
            await handleRefund(event.data.object as Stripe.Charge);
            break;

            default:
              console.log(`Unhandled event type: ${event.type}`);
    }

    return NextResponse.json({ received: true });
  } catch (error) {
    console.error('Webhook handler failed:', error);
    return NextResponse.json({ error: 'Handler failed' }, { status: 500
});
  }
}
```



```

});
}
}

async function handlePaymentSuccess(paymentIntent: Stripe.PaymentIntent) {
  const invoiceId = Number(paymentIntent.metadata.invoiceId);
  const amount = paymentIntent.amount / 100; // Convert from cents

  await db.transaction(async (tx) => {
    // Record transaction
    await tx.insert(paymentTransactions).values({
      invoiceId,
      amount: amount.toString(),
      currency: paymentIntent.currency.toUpperCase(),
      paymentMethod: 'card',
      provider: 'stripe',
      providerTransactionId: paymentIntent.id,
      status: 'completed',
    });

    // Update invoice
    const invoice = await tx.query.invoices.findFirst({
      where: eq(invoices.id, invoiceId),
    });

    if (invoice) {
      const newPaidAmount = Number(invoice.paidAmount || 0) + amount;
      const newStatus = newPaidAmount >= Number(invoice.total) ? 'paid' :
      'unpaid';

      await tx
        .update(invoices)
        .set({
          paidAmount: newPaidAmount.toString(),
          status: newStatus,
        })
        .where(eq(invoices.id, invoiceId));
    }
  });
}

```

Webhook Testing:

```

# Install Stripe CLI
stripe login

# Forward webhooks to local server
stripe listen --forward-to localhost:3000/api/webhooks/stripe

# Trigger test events
stripe trigger payment_intent.succeeded

```

[!Warning]

- Stripe will retry failed webhooks up to 3 times
- Implement idempotency to handle duplicate events
- Store webhook events in database for audit trail
- Monitor webhook health in Stripe Dashboard

IX. Testing Strategy

A. Unit Tests (Vitest)

- **Example:** Test `calculateRidePrice`:

```
test("calculates ride price with options and waiting time", () => {  
  const result = calculateRidePrice(10, 5, [{ name: "vip", price: 5  
}], 10);  
  expect(result.total).toBe(27.5); // 10 + 7.5 + 5 + 5 = 27.5  
});
```

[!Note]

- Unit tests cover individual functions and business logic.
- All tests must pass before code can be merged.

B. Integration Tests

- **Tools:** Vitest + Drizzle/Neon mocks.
- **Scenarios:**

```
flowchart LR  
  A[Create Ride] --> B[Assign Driver] --> C[Generate Invoice]
```

[!Note]

- Integration tests verify that different components work together correctly.
- Database interactions are mocked to ensure test reliability and speed.

C. End-to-End Tests

- **Tools:** Playwright.
- **Scenarios:**
 - Full user journey (booking, payment, rating).

[!Note]

- End-to-end tests simulate real user interactions from start to finish.
- Critical paths (booking, assignment, payment) are tested thoroughly.
- Tests run in isolated environments to prevent interference with production data.

X. Deployment & CI/CD

A. Environment Variables

Required configuration for all environments:

Variable	Description	Example	Required
DATABASE_URL	Neon PostgreSQL connection string	postgresql://user:pass@host/db	Yes
AUTH_SECRET	better-auth secret key	Random 32-char string	Yes
AUTH_URL	Application URL for auth redirects	https://amt-horizon.com	Yes
STRIPE_SECRET_KEY	Stripe API secret key	sk_live_...	Yes
STRIPE_PUBLISHABLE_KEY	Stripe publishable key	pk_live_...	Yes
STRIPE_WEBHOOK_SECRET	Stripe webhook signing secret	whsec_...	Yes
MAPBOX_ACCESS_TOKEN	Mapbox API token for geolocation	pk.eyJ...	Yes
BLOB_READ_WRITE_TOKEN	Vercel Blob storage token	vercel_blob_...	Yes
OCR_API_KEY	Tesseract.js or cloud OCR API key	Optional for client-side OCR	No
EMAIL_SERVER	SMTP server for notifications	smtp://...	Yes
EMAIL_FROM	Sender email address	noreply@amt-horizon.com	Yes

Variable	Description	Example	Required
SENTRY_DSN	Sentry error tracking DSN	https://...@sentry.io/...	No
NEXT_PUBLIC_APP_URL	Public-facing app URL	https://amt-horizon.com	Yes

[!Note]

- Store secrets securely using environment variables, never commit to git.
- Use different keys for development, staging, and production.
- Rotate secrets regularly for security.

B. Environments

Environment	URL	Database	Purpose
Development	localhost:3000	Neon (Dev)	Local development
Staging	staging.amt-horizon.com	Neon (Staging)	Pre-production testing
Production	amt-horizon.com	Neon (Prod)	Live application

C. CI/CD Pipeline

Pipeline Steps:

1. **Run Tests:** Execute unit, integration, and e2e tests on every PR
2. **Lint Code:** Ensure code quality standards with ESLint
3. **Run Database Migrations:** Apply schema changes using `drizzle-kit push`
4. **Build Application:** Create production build and verify no errors
5. **Deploy to Staging:** Auto-deploy on merge to `develop` branch
6. **Manual Approval:** Require approval before production deployment
7. **Deploy to Production:** Deploy after approval on merge to `main`
8. **Health Check:** Verify deployment success with smoke tests

[!Note]

- Database migrations run before deployment to ensure schema is updated.
- Use `drizzle-kit push` for automatic migrations in development/staging.
- For production, consider manual migration review process for safety.

D. Rollback Strategy

Application Rollback:

```
# Revert to previous Vercel deployment
vercel rollback [deployment-url]
```

Database Rollback:

```
# Create backup before migrations
pg_dump $DATABASE_URL > backup_$(date +%Y%m%d_%H%M%S).sql

# Restore from backup if needed
psql $DATABASE_URL < backup_20260119_143000.sql
```

[!Warning]

- Test rollback procedures in staging before production.
- Document all schema changes in migration files.
- Keep backups for at least 30 days.

E. Monitoring & Logging

Error Tracking: Sentry for runtime errors and performance monitoring

Performance Monitoring:

- Vercel Analytics for Web Vitals and page performance
- Database query performance via Neon dashboard
- API response times via Vercel logs

Logging Best Practices:

- Use structured logging for better querying
- Log levels: error, warn, info, debug
- Never log sensitive data (passwords, tokens, PII)
- Retain logs for 30 days minimum for auditing